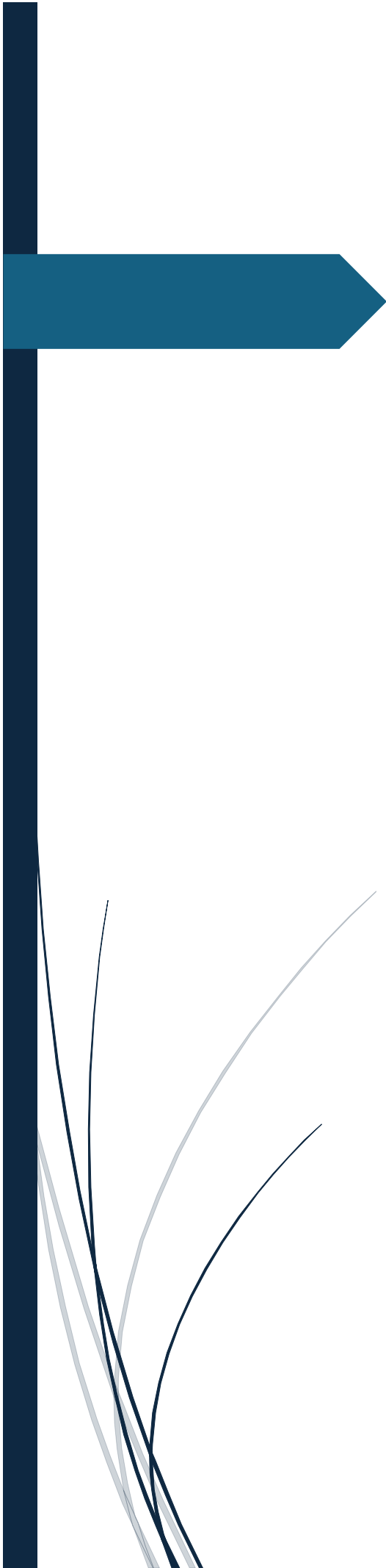




Proyecto final DAW

Memoria proyecto BsnessApp

Fu Jun Fu Lin

GRADO SUPERIOR DESARROLLO DE APLICACIONES WEB

Contenido

Índice de ilustraciones	2
1. Introducción	3
1.1 Componente TPV	3
2. Estructura del proyecto	4
3. Características técnicas.....	5
3.1 Despliegue de la aplicación	6
3.2 Otras dependencias	6
3.2.1 Generador de pdfs – React-pdf.....	6
3.2.2 Printer Server	10
3.2.3 Sistema de mailing	11
4. Características de la base de datos	14
4.1 Descripción de las tablas	15
Tabla: users	15
Tabla: categories.....	15
Tabla: taxes	16
Tabla: articles	16
Tabla: receipts-numbers	16
Tabla: payment-methods	16
Tabla: receipts	17
Tabla: receipts-lines.....	17
Tabla: end-days	18
Vista: article_view.....	18
Vista: receipt_view.....	18
5. Guía de estilo	19
5.1 Introducción.....	19
Stack técnico.....	19
5.2 Color y uso tipográfico del color	20
5.2.1 Blue – Acciones primarias y tema claro	20
5.2.2 Stone – Colores neutros y tema claro	20
5.2.3 Slate – Colores neutros y tema oscuro.....	20
5.2.4 Cyan – Acciones primarias y tema oscuro.....	21
5.2.5 Colores semánticos	21
5.3 Tipografía.....	22
5.4. Iconografía	22
Configuración.....	23

Listado de iconos usados.....	23
Tamaños	24
Muestras	24
5.5 Estructura de la web. Maquetación - Wireframes	25
5.5.1 Dashboard	25
5.5.2 Pos	26
5.6 Espaciados y layout	27
5.6.1 Contenedores	27
5.6.2 Espaciado	27
6. Caso de uso	28
7. Anexos.....	28
7.1 Anexo – Diagrama de clase.....	28
7.2 Anexo – Diagrama entidad relación.....	28
7.3 Anexo - CU-001_Registrar_Venta_POS	28
8. Bibliografía.....	29

Índice de ilustraciones

Ilustración 1. Diagrama de uso de periféricos.....	4
Ilustración 2. Ejemplo ticket en pdf	9
Ilustración 3. Ejemplo ticket impreso	11
Ilustración 4. Ejemplo email enviado al cliente.....	14
Ilustración 5. Wireframe lowfidelity "/dashboard"	25
Ilustración 6. Pantalla final "/dashboard"	25
Ilustración 7. Wireframe lowfidelity "/pos"	26
Ilustración 8. Pantalla final "/pos"	26

1. Introducción

BsnessApp es una aplicación SaaS o aplicación como servicio en el cual el cliente puede gestionar un terminal de punto de venta (TPV) o Point Of Sale (POS) en inglés con todos los periféricos necesarios para llevar a cabo la tarea y poder observar, modificar y extraer informes de la actividad diaria de la empresa. Las funciones principales que pueden hacer los usuarios son:

- El usuario puede generar recibos o facturas simplificadas y cobrar los ha sus clientes.
- Enviar el recibo a los clientes vía email.
- Uso de periféricos como un lector de código de barras o una impresora térmica para imprimir el recibo.
- Obtener datos de ventas con diferentes filtros de datos.
- Crear informes de venta y exportarlos.
- Crear artículos.
- Acceso a la aplicación según rol en la empresa.

El proyecto se encuentra alojado en **Github** en el siguiente enlace:

https://github.com/fumetria/DAW_Proyecto_Final

El proyecto se encuentra desplegado en **Vercel** en el siguiente enlace:

<https://bsnessapp.es>

1.1 Componente TPV

El TPV es uno de los componentes más importantes de nuestra aplicación. Los clientes conectándose a nuestra aplicación pueden usar el TPV conectado a su organización, es decir, pueden trabajar con los datos de su organización (artículos, facturación, entre otros) e interactuar con sus periféricos de TPV que disponen (impresoras térmicas, cajones portamonedas, etc.).

Interactuando con los periféricos

Para poder trabajar con periféricos, instalamos al cliente un servidor local donde mediante endpoints del servidor podemos interactuar con la impresora térmica que tenga instalada.

Lo ideal para el negocio sería proveer al cliente un lote de hardware con un monitor todo en uno, impresora térmica, cajón portamonedas y lector de código de barras con toda la configuración local instalada.

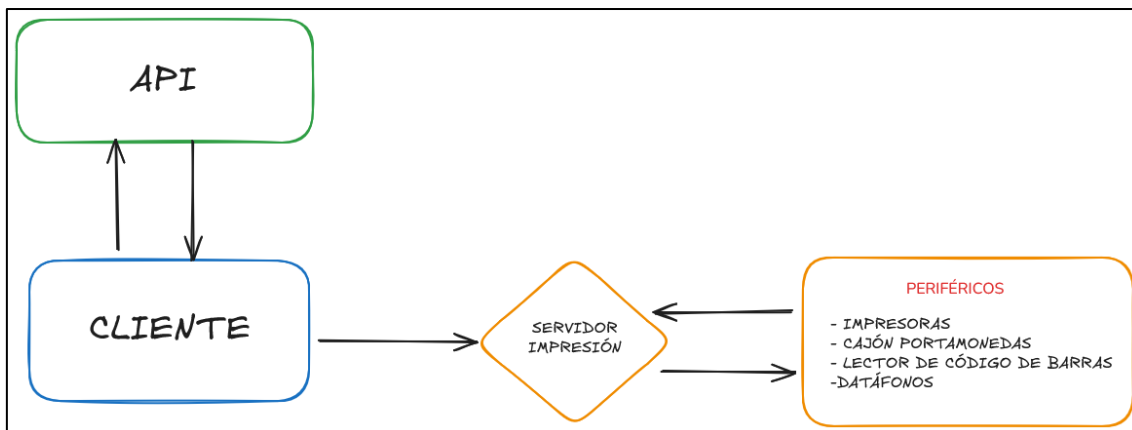


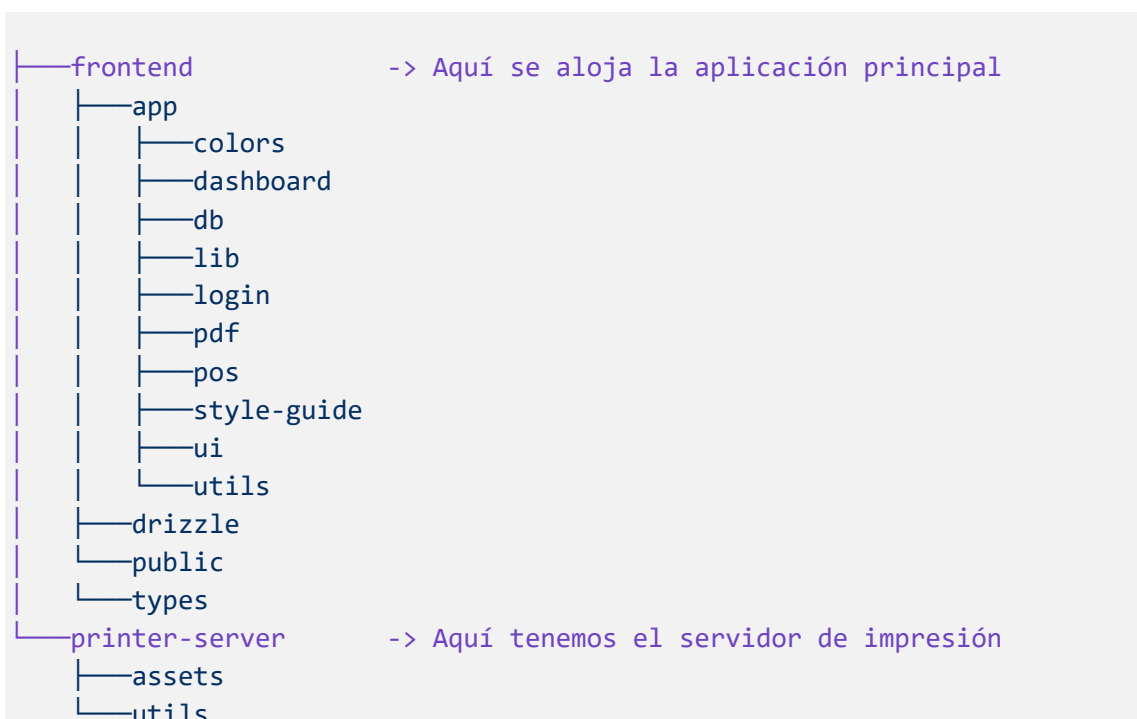
Ilustración 1. Diagrama de uso de periféricos

Por motivos de seguridad, los navegadores web no pueden acceder a los puertos locales del cliente de manera nativa, excepción de Chrome que se puede dar permisos especiales a esta función, es por eso, que mediante un middleware como servicio de impresión podemos interactuar con los periféricos.

Usando Node como servicio de backend (printer-server), podemos acceder a los puertos que nos interesan de la máquina del cliente y poder interactuar con él, en nuestro caso, el puerto COM6 donde tenemos virtualizado la impresora térmica. Con esta configuración, solo nos hace falta configurar ciertos endpoints para que realice las tareas que le indicamos como imprimir un ticket o abrir el cajón portamonedas. Todo esto lo explicamos con más detalle en el apartado de dependencias del proyecto.

2. Estructura del proyecto

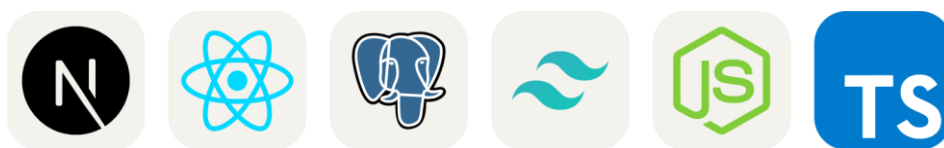
El proyecto está estructurado de la siguiente forma:



La aplicación principalmente funciona desde la carpeta frontend ya que Next.js usa una arquitectura monolítica y hace de frontend y backend desde un mismo directorio.

Desde la carpeta *printer-server* podemos ejecutar nuestro servidor de impresión en nuestro equipo y este escuchará todas las peticiones desde nuestra aplicación para imprimir los recibos que le enviemos.

3. Características técnicas



Para la creación de nuestra aplicación hemos usado las siguientes tecnologías:

- **Next.js:** Framework que permite crear apps web completas (frontend y backend) con React de forma sencilla.
- **React:** Librería para construir interfaces de usuario dinámicas usando componentes reutilizables.
- **PostgreSQL:** Base de datos que guarda y organiza la información de forma estructurada. Para el uso de esta base de datos hemos usado **NeonDB** como sistema de gestión de bases de datos ya que su arquitectura **Serverless** nos garantiza un escalado automático y eficiente de los recursos. Su capacidad de **Database Branching** permite una integración fluida con el flujo de trabajo de Vercel, habilitando entornos de prueba aislados y seguros. Además, su optimización para conexiones en entornos de ejecución **Edge** asegura una baja latencia y alta disponibilidad, fundamentales para la arquitectura moderna de Next.js.
- **Drizzle ORM:** Herramienta que facilita trabajar con la base de datos usando código en lugar de SQL directo. Hemos implementado **Drizzle ORM** como capa de acceso a datos debido a su enfoque **Headless** y su mínima huella de computación, lo cual es crítico para mantener la eficiencia en entornos **Serverless**. Su capacidad para ofrecer seguridad de tipos completa (**Type Safety**) y su compatibilidad nativa con drivers sobre HTTP/WebSockets permiten una comunicación ultra-rápida entre las funciones de **Vercel** y la base de datos **Neon**, garantizando un rendimiento óptimo y un mantenimiento del código más robusto.
- **Tailwind CSS:** Framework de estilos que permite diseñar interfaces rápidamente con clases predefinidas.
- **Node.js:** Entorno que permite ejecutar JavaScript en el servidor para manejar lógica y conexiones con hardware. Esto es lo que usaremos para gestionar principalmente nuestra impresora térmica de manera local.

3.1 Despliegue de la aplicación

La aplicación la hemos desplegado usando los servicios de Vercel ya que Next.js ha sido creado por Vercel y este ofrece la implementación de referencia del framework, garantizando que cada funcionalidad nueva sea compatible desde el "día cero".

Vercel está diseñado específicamente para entender la estructura de Next.js. Mientras que en otros proveedores (como AWS o GCP) tendrías que configurar manualmente servidores, contenedores o procesos de construcción, Vercel:

- Detecta automáticamente la configuración del proyecto.
- Optimiza el proceso de **Build** sin intervención manual.
- Gestiona de forma transparente las variables de entorno y los prefijos de seguridad.

Otra de la ventaja de usar Vercel, es que tiene aplicaciones integradas como es el sistema de bases de datos Neon DB que es el que usamos en nuestra aplicación. En nuestro caso lo hemos seleccionado como sistema de base de datos por los siguientes motivos:

- La versión gratuita tiene un servicio muy amplio de uso que en nuestro caso nos sobra.
- Type Safety: Se integra perfectamente con ORMs modernos como Prisma o Drizzle, facilitando la generación de tipos TypeScript a partir del esquema de Neon.
- Despliegue simplificado: La integración oficial con Vercel permite importar las variables de entorno (DATABASE_URL) con un par de clics, automatizando la conexión entre el frontend y el backend.

Por otro lado, tenemos comprado el dominio *bsnessapp.es* a **Ionos Cloud** donde hemos redireccionado este dominio a nuestra aplicación en Vercel. Además, hemos añadido en nuestro servicio de Vercel todas las variables de DNS de Ionos para poder usar nuestro email corporativo para enviar los tickets en pdf a nuestros clientes desde la aplicación.

3.2 Otras dependencias

3.2.1 Generador de pdfs – React-pdf

React-pdf es una librería diseñada para crear archivos PDF utilizando componentes de React. A diferencia de otras soluciones que realizan capturas de pantalla de elementos HTML (DOM-to-PDF), React-pdf renderiza directamente en el formato PDF, lo que permite un control total sobre el documento, archivos más ligeros y texto seleccionable de alta calidad. Estos pdf se pueden generar en el servidor o en el cliente, en nuestro caso, los generamos en el cliente, pues no guardamos ningún pdf en nuestro servidor. Cuando el usuario ejecuta una acción donde interviene el pdf, este se genera al momento para mostrárselo al cliente o, en su caso, enviar el pdf al email indicado.

Al integrar esta dependencia, el proyecto gana la capacidad de generar reportes dinámicos, facturas o informes directamente desde los datos de la aplicación, manteniendo la consistencia visual y la facilidad de mantenimiento que ofrece el desarrollo basado en componentes. Actualmente, solo se pueden generar tickets o informes de fin de día.

A continuación, tenemos un ejemplo del propio proyecto:

ReceiptPDF.tsx

Este código es lo que posteriormente se traduciría en el ticket en pdf que enviamos a los clientes.

```
<Document>
  <Page size={[pageWidth, pageHeight]} style={styles.page}>
    <View style={styles.center}>
      <Text style={[styles.headerSub,
styles.bold]}>Cafeteria</Text>
      <Text style={[styles.headerTitle,
styles.bold]}>L&apos;ESTACIÓ</Text>
      <View style={styles.headerLogo}>
        {/* eslint-disable-next-line jsx-a11y/alt-text -- @react-
pdf Image has no alt prop */}
        <Image src={logoDataUrl} style={{ width: 75, height: 50
}} />
      </View>
      <Text style={styles.headerSub}>Ctra. l&apos;estació
S/N</Text>
      <Text style={styles.headerSub}>Tel: 96 291 93 75</Text>
      <Text style={styles.headerSub}>Email:
46006100@edu.gva.es</Text>
    </View>
    <View style={styles.line} />
    {/* Receipt date and number section */}
    <View style={[styles.center, styles.mbMD]}>
      <Text>FACTURA SIMPLIFICADA</Text>
    </View>
    <View style={styles.row}>
      <Text style={styles.left}>{dateStr}</Text>
      <Text style={styles.right}>{receipt.num_receipt}</Text>
    </View>
    <View style={styles.line} />

    {/* Receipt lines content section */}
    <View style={styles.tableHeader}>
      <Text style={[styles.colCant,
styles.tableHeaderCell]}>Cant</Text>
      <Text style={[styles.colName,
styles.tableHeaderCell]}>Nombre</Text>
      <Text style={[styles.colPrice,
styles.tableHeaderCell]}>Pre.</Text>
      <Text style={[styles.colTotal,
styles.tableHeaderCell]}>Total</Text>
    </View>
    {receipt.lines.map((line) => (
      <View key={line.id} style={styles.tableRow}>
        <Text style={styles.colCant}>{line.quantity}</Text>
        <Text style={styles.colName}>
```



```

    {(line.article_name ?? "-").toUpperCase().slice(0,
25))}
    </Text>
    <Text
style={styles.colPrice}>{formatPrice(line.price)}</Text>
    <Text
style={styles.colTotal}>{formatPrice(line.total)}</Text>
    </View>
  )))

  { /* Total receipt price section */}
  <View style={styles.line} />
  {(baseTotal != null || taxTotal != null) && (
    <>
      <View style={[styles.row, styles.textSmall]}>
        <Text>Base imponible</Text>
        <Text style={styles.right}>
{baseTotal != null ? `${formatPrice(baseTotal)} €` : "-"}
        </Text>
      </View>
      <View style={[styles.row, styles.textSmall]}>
        <Text>IVA</Text>
        <Text style={styles.right}>
{taxTotal != null ? `${formatPrice(taxTotal)} €` : "-"}
        </Text>
      </View>
    </>
  )}
  <View style={[styles.row, styles.totalLine]}>
    <Text style={styles.bold}>Total (Impostos Incl.)</Text>
    <Text style={[styles.right, styles.bold]}>
      {formatPrice(receipt.total)} €
    </Text>
  </View>
  <View style={[styles.row, styles.textSmall]}>
    <Text>M. pagament: {receipt.payment_method}</Text>
    <Text />
  </View>
  <View style={styles.footer}>
    <Text>
      Ha sigut atès per{" "}
      <Text style={styles.bold}>{userName ?? "Empleado"}</Text>
    </Text>
    <View style={[styles.center, styles.footerQrWrap]}>
      { /* If we want to show some QR code, put inside this View
like company website url */}
      { /* eslint-disable-next-line jsx-a11y/alt-text -- @react-
pdf Image has no alt prop */}

```

```

    <Image src={qrDataUrl} style={{ width: 50, height: 50 }} />
  </View>
  <Text>Gracies per la seua visita</Text>
</View>
</Page>
</Document>

```

El pdf sería este:

Cafeteria
L'ESTACIÓ



Ctra. l'estació S/N
Tel: 96 291 93 75
Email: 46006100@edu.gva.es

FACTURA SIMPLIFICADA

30/04/2026 7:17

FS26-000144

Cant	Nombre	Pre.	Total
1	ESPECIAL DE LA CASA	6,50	6,50
1	BLANCO Y NEGRO	6,00	6,00
1	SERRANITO	6,00	6,00

Base imponible

IVA

Total (Impostos Incl.)

M. pagament: Efectivo

16,81 €

1,69 €

18,50 €

Ha sigut atès per **Admin**



Gracies per la seua visita

Ilustración 2. Ejemplo ticket en pdf

3.2.2 Printer Server

El *printer-server* es un servicio de impresión para impresoras térmicas mediante un servidor de Node.js. El servidor permite trabajar con equipos POS, como impresoras térmicas y cajas registradoras, a través de una interfaz HTTP sencilla.

La arquitectura permite que una aplicación frontend alojada en la nube se comunique con el hardware POS local a través de este servidor intermedio. En nuestro caso específico, se utiliza una impresora EPSON TM-T20II configurada mediante un puerto COM virtual, asignado utilizando la aplicación EPSON TM Virtual Port Driver Port proporcionada por EPSON.

Dependencias:

- **Node.js:** Node.js es un entorno de ejecución de JavaScript gratuito, de código abierto y multiplataforma que permite a los desarrolladores crear servidores, aplicaciones web, herramientas de línea de comandos y scripts.
- **Express:** Express es un framework minimalista y flexible para aplicaciones web Node.js que proporciona un conjunto robusto de características para aplicaciones web y móviles.
- **node-thermal-printer:** Módulo Node.js para impresión por línea de comandos en impresoras térmicas Epson, Star, Tanca, Drauma, Custom y Brother.
- **serialport:** Dependencia de JavaScript para acceder a puertos en serie del sistema operativo.

Uso

Abrimos nueva terminal, copiamos nuestras variables de entorno y ejecutamos el `index.js` para levantar el servidor de impresión en `localhost`:

```
cd printer-server
cp .env.example .env
node index.js
```

- Si tenemos configurada correctamente la impresora térmica al lanzar a la ruta `/print` nos debería salir el ticket con los datos que le hemos enviado.
- Para abrir el cajón, enviamos información a la ruta `/open-drawer`.



Ilustración 3. Ejemplo ticket impreso

3.2.3 Sistema de mailing

Como alternativa a la impresión en papel, hemos creado un sistema que envía el ticket al cliente si se introduce el email del cliente. Este sistema genera un pdf con los datos del ticket al finalizar un ticket y lo manda por email al email indicado. Como hemos indicado anteriormente, usamos los datos que nos proporciona lonos para poder enviar nuestro email, para ellos configuramos las variables en nuestro archivo de configuración **.env** que tenemos a continuación:

```
# Variables de entorno para el envío de email
EMAIL_HOST=smtp.domain.com
EMAIL_PORT=465
EMAIL_USER=user@example.es
EMAIL_PASSWORD=mailpassword #Constraseña del email o contraseña de
aplicación
# Datos de email de origen
EMAIL_FROM=user@example.es
```

```
EMAIL_FROM_NAME="Example Name Company"
```

Dependencias:

- [Nodemailer](#): Librería para Node.js que gestiona todo el proceso para el envío del email.
- [React-pdf](#): Dependencia de React para la generación de pdf y visualización en el navegador mediante componentes del Framework React.

El sistema de mailing consta de dos partes, por un lado tenemos el archivo mail.ts donde tenemos toda la lógica para enviar el email y por otro lado, tenemos nuestro servicio de generación de pdf que hemos explicado en el apartado anterior.

Nodemailer es una librería muy completa para el tratamiento de datos y envío de emails. Una de las grandes capacidades que tiene es que podemos enviar una página html como email y adjuntar archivos al email enviado. A continuación, mostraremos trozos de código de nuestro script de email:

Función creación del email

```
function buildReceiptEmailHtml(
  to: string,
  receipt: ReceiptDetailForEmail
): string {
  const dateStr = formatDate(receipt.created_at);
  const totalStr = formatPrice(receipt.total);
  return `
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1"
/>
  <title>Tu ticket digital</title>
</head>

<head>
  <style>
    .body {
      margin: 0;
      padding: 0;
      background: #f5f6f8;
      font-family: Arial, Helvetica, sans-serif;
    }
  </style>
</head>
```

```

<div class="body">
  <div class="container">
    <div class="header">
      
      <strong>Cafeteria L'Estació</strong>
    </div>
    .
    .
    .
    </div>
    <div class="footer2">
      
    </div>
  </div>
</body>
</html>
`.trim();
}

```

Función enviar email

```

await transport.sendMail({
  from: sender,
  to,
  subject,
  html,
  attachments: [
    { #Aquí adjuntamos el pdf creado
      filename,
      content: pdfBuffer,
      contentType: "application/pdf",
    },
    {
      filename: "iestacio_logo.png",
      content: await readFile(path.join(process.cwd(), "app", "pdf",
"components", "iestacio_logo.png")),
      cid: "iestacio_logo",
    },
    {
      filename: "iestacio_mail_footer.png",
      content: await readFile(path.join(process.cwd(), "app", "pdf",
"components", "iestacio_mail_footer.png")),
      cid: "iestacio_mail_footer",
    }
  ],
});

```

Email final:

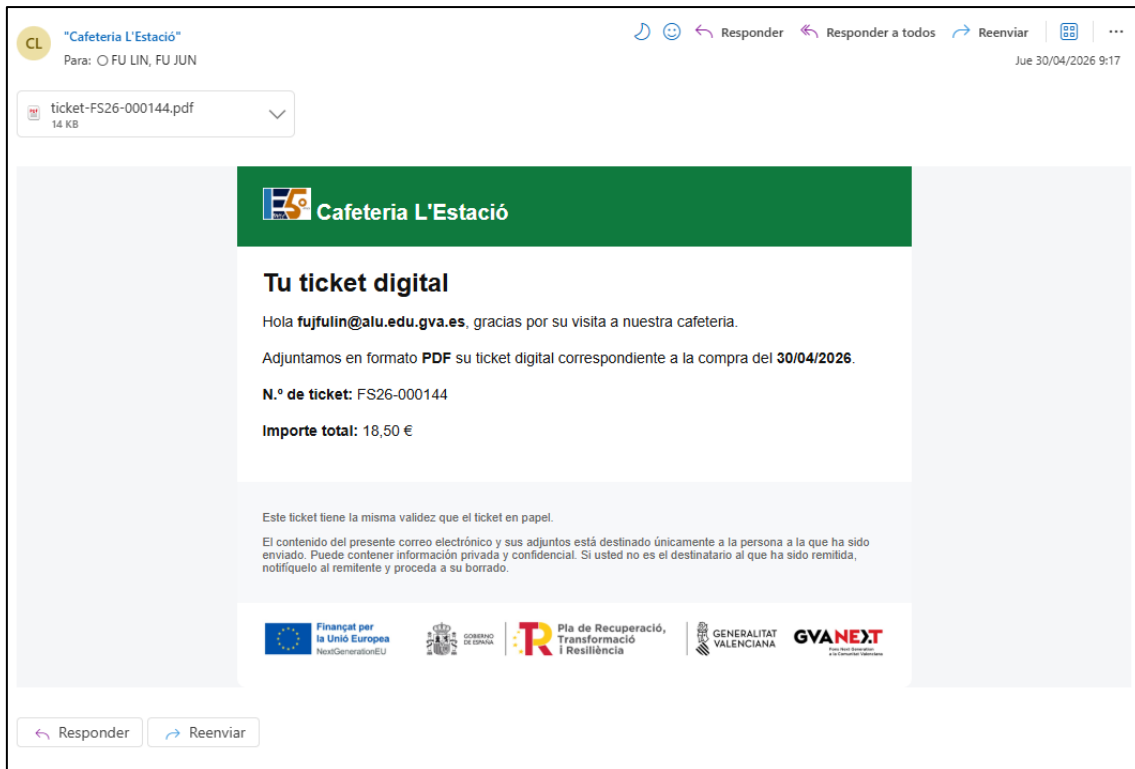


Ilustración 4. Ejemplo email enviado al cliente

4. Características de la base de datos

En este proyecto como base de datos me he decantado por PostgreSQL ya que es una de las bases de datos relacionales de código abierto más potentes y fiables, elegida por su alta integridad de datos, escalabilidad y compatibilidad con estándares SQL. Cuenta con soporte comunitario extenso y actualizaciones constantes que garantiza su evolución a largo plazo.

Para la gestión de la base de datos usaremos Drizzle ORM que es herramienta que nos facilita la gestión de la base de datos haciendo más simple hacer consultas y gestionar la base de datos. Por ejemplo:

Sql

```
SELECT * FROM users WHERE name = 'Juan';
```

Drizzle ORM - Typescript

```
await db.select().from(users).where(eq(users.name, 'Juan'));
```

Aunque parece más largo usando Drizzle ORM, hay unas diferencias clave:

- **Postgres (SQL):** Es una cadena de texto (un "string"). Tu editor no sabe si lo que escribiste dentro tiene sentido hasta que el código corre.
- **Drizzle:** Son funciones reales. Tienes autocompletado, por lo que al escribir *users*, el asistente te mostrará todas las opciones disponibles (nombre, email, edad, etc.) con todo el ahorro de tiempo que conlleva.

4.1 Descripción de las tablas

Campos comunes a todas las tablas

Todas las tablas incluyen los campos heredados: `created_at` (TIMESTAMP NOT NULL, default NOW()), `updated_at` (TIMESTAMP, nullable), `deleted_at` (TIMESTAMP, nullable — soft-delete).

Tabla: users

Almacena los usuarios del sistema (empleados / administradores).

Campo	Tipo	Restricciones / Default	Descripción
id	<i>uuid</i>	PK, default <code>gen_random_uuid()</code>	Identificador único
email	<i>varchar(255)</i>	NOT NULL, UNIQUE	Correo electrónico
password	<i>varchar(255)</i>	NOT NULL	Contraseña cifrada
dni	<i>varchar(12)</i>	NOT NULL, UNIQUE	DNI del usuario
name	<i>varchar(255)</i>	NOT NULL	Nombre
surname1	<i>varchar(255)</i>	NOT NULL	Primer apellido
surname2	<i>varchar(255)</i>	Nullable	Segundo apellido
rol	<i>varchar(50)</i>	NOT NULL, default 'user'	Rol: 'user' 'admin'
is_active	<i>boolean</i>	Nullable, default true	Usuario activo/inactivo

Relaciones:

- *receipts.user_email* → *users.email*
- *end-days.user_email* → *users.email*

Tabla: categories

Catálogo de categorías para clasificar artículos.

Campo	Tipo	Restricciones / Default	Descripción
id	<i>integer</i>	PK, GENERATED ALWAYS AS IDENTITY	Identificador único
name	<i>varchar(255)</i>	NOT NULL, UNIQUE	Nombre de la categoría

Relaciones:

- *articles.category_id* → *categories.id*

Tabla: taxes

Tipos impositivos (IVA u otros impuestos) aplicables a artículos.

Campo	Tipo	Restricciones / Default	Descripción
id	integer	PK, GENERATED ALWAYS AS IDENTITY	Identificador único
value	real	NOT NULL, UNIQUE	Valor del impuesto (%)

Relaciones:

- *articles.tax_id* → *taxes.id*
- *receipts-lines.tax_id* → *taxes.id*

Tabla: articles

Productos o servicios vendibles del negocio.

Campo	Tipo	Restricciones / Default	Descripción
id	uuid	PK, default gen_random_uuid()	Identificador único
cod_art	varchar(255)	NOT NULL, UNIQUE	Código de artículo
name	varchar(255)	NOT NULL	Nombre del artículo
category_id	integer	NOT NULL, FK → categories.id	Categoría
tax_id	integer	Nullable, FK → taxes.id	Impuesto aplicado
pvp_without_tax	real	Nullable	Precio sin impuesto
pvp	real	NOT NULL	Precio de venta al público
is_active	boolean	NOT NULL, default true	Artículo activo/inactivo

Relaciones:

- *receipts-lines.article_cod_art* → *articles.cod_art*

Tabla: receipts-numbers

Contador de numeración de tickets/facturas por serie y año.

Campo	Tipo	Restricciones / Default	Descripción
id	integer	PK, GENERATED ALWAYS AS IDENTITY	Identificador único
serie	varchar(20)	NOT NULL, default 'FS'	Serie del documento
year	integer	NOT NULL, default año actual (2 dígitos)	Año del documento
number	integer	NOT NULL, default 0	Último número emitido

🔗 Índice único compuesto: (serie, year).

Tabla: payment-methods

Métodos de pago aceptados (efectivo, tarjeta, etc.).

Campo	Tipo	Restricciones / Default	Descripción
id	integer	PK, GENERATED ALWAYS AS IDENTITY	Identificador único

Campo	Tipo	Restricciones / Default	Descripción
name	varchar(255)	NOT NULL, UNIQUE	Nombre del método de pago

Relaciones:

- *receipts.payment_method* → *payment-methods.id*

Tabla: receipts

Cabecera de tickets / facturas de venta.

Campo	Tipo	Restricciones / Default	Descripción
id	uuid	PK, default gen_random_uuid()	Identificador único
num_receipt	varchar	NOT NULL, UNIQUE	Número completo del ticket (serie+año+nº)
serie	varchar(20)	NOT NULL	Serie del documento
year	integer	NOT NULL	Año del documento
number	integer	NOT NULL	Número secuencial
base_total	real	NOT NULL, default 0	Base imponible total
tax_total	real	NOT NULL, default 0	Total de impuestos
total	real	NOT NULL, default 0	Total con impuestos
user_email	varchar	NOT NULL, FK → users.email	Usuario que emite
payment_method	integer	NOT NULL, FK → payment-methods.id	Método de pago
is_open	boolean	NOT NULL, default true	Ticket abierto / cerrado
end_day_id	uuid	Nullable	Referencia al cierre de caja

Relaciones:

- *receipts-lines.receipt_id* → *receipts.num_receipt* (ON DELETE CASCADE)
- *end-days.first_receipt_id* → *receipts.num_receipt*
- *end-days.last_receipt_id* → *receipts.num_receipt*

Tabla: receipts-lines

Líneas de detalle de cada ticket (artículos vendidos).

Campo	Tipo	Restricciones / Default	Descripción
id	uuid	PK, default gen_random_uuid()	Identificador único
article_cod_art	varchar	NOT NULL, FK → articles.cod_art	Artículo vendido
details	varchar(255)	Nullable	Descripción adicional
quantity	integer	NOT NULL, default 0	Cantidad
price	real	NOT NULL	Precio unitario
base_unit	real	Nullable	Base imponible unitaria
tax_unit	real	Nullable	Impuesto unitario
base_total	real	Nullable	Base imponible línea
tax_total	real	Nullable	Impuesto línea
tax_id	integer	Nullable, FK → taxes.id	Tipo de impuesto aplicado
tax_value	real	Nullable	Valor % del impuesto en el momento de venta

Campo	Tipo	Restricciones / Default	Descripción
total	<i>real</i>	NOT NULL	Total de la línea
receipt_id	<i>varchar</i>	NOT NULL, FK → receipts.num_receipt, ON DELETE CASCADE	Ticket al que pertenece

Tabla: end-days

Registro de cierres de caja diarios.

Campo	Tipo	Restricciones / Default	Descripción
id	<i>uuid</i>	PK, default gen_random_uuid()	Identificador único
date	<i>varchar</i>	NOT NULL	Fecha del cierre
base_total	<i>real</i>	NOT NULL, default 0	Base imponible total del día
tax_total	<i>real</i>	NOT NULL, default 0	Total impuestos del día
total	<i>real</i>	NOT NULL	Total del día con impuestos
first_receipt_id	<i>varchar</i>	NOT NULL, FK → receipts.num_receipt	Primer ticket del día
last_receipt_id	<i>varchar</i>	NOT NULL, FK → receipts.num_receipt	Último ticket del día
total_receipts	<i>integer</i>	NOT NULL	Nº total de tickets del día
user_email	<i>varchar</i>	NOT NULL, FK → users.email	Usuario que realiza el cierre

Vista: article_view

Vista de artículos donde devuelve los datos del artículo con el **nombre de la categoría** ya resuelto (en lugar del category_id numérico).

Columna	Tipo	Descripción
articleID	<i>uuid</i>	ID del artículo
articleCOD	<i>varchar</i>	Código del artículo
articleName	<i>varchar</i>	Nombre del artículo
articleCategory	<i>varchar</i>	Nombre de la categoría (JOIN)
articlePvp	<i>real</i>	Precio de venta al público
articleIsActive	<i>boolean</i>	Estado activo/inactivo

Relaciones:

➤ *articles* → LEFT JOIN → *categories*: *articles.category_id* = *categories.id*

Vista: receipt_view

Vista de tickets donde devuelve los datos del ticket con el **nombre del método de pago** ya resuelto (en lugar del id numérico).

Columna	Tipo	Descripción
id	<i>uuid</i>	ID del ticket
num_receipt	<i>varchar</i>	Número del ticket
created_at	<i>timestamp</i>	Fecha/hora de creación
total	<i>real</i>	Total del ticket

Columna	Tipo	Descripción
payment_method	varchar	Nombre del método de pago (JOIN)
user_email	varchar	Email del usuario
is_open	boolean	Estado abierto/cerrado

Relaciones:

- *receipts* → *LEFT JOIN* → *payment_methods: receipts.payment_method = payment-methods.id*

En ambas views se usa *LEFT JOIN*, lo que significa que si el registro relacionado no existe (por ejemplo, un artículo sin categoría asignada o un ticket sin método de pago), la fila igualmente aparece en la vista con ese campo como *null*.

5. Guía de estilo

5.1 Introducción

Esta guía define los estándares visuales para la aplicación BsnessApp, asegurando una experiencia de usuario coherente y profesional. Al utilizar Tailwind CSS, la implementación se basa en un sistema de diseño atómico mediante clases de utilidad. Ejemplo:

```
<Link
  title="Nuevo artículo"
  href={` /dashboard/maintance/articles/create/`}
  className="flex items-center gap-2 py-2 px-3 rounded-xl bg-blue-500
text-stone-100 border border-blue-500 hover:bg-blue-200 hover:border-
blue-500 hover:text-blue-500  dark:bg-cyan-500 dark:text-slate-50
dark:border-cyan-500 dark:hover:bg-cyan-200 dark:hover:text-cyan-500
dark:hover:border-cyan-500"
>
  <p className="hidden md:block">Nuevo Artículo</p>
  <FontAwesomeIcon icon={faPlus} />
</Link>
```

Como podemos observar en este componente, todos los estilos desde colores a comportamiento como contenedor esta estilado en las clases de los elementos HTML.

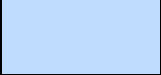
Stack técnico

Tecnología	Versión	Uso
Next.js	16.x	Framework React
Tailwind CSS	V4	Utilidades CSS
Font Awesome	7.x	Iconografía
clsx	2.x	Clases condicionales

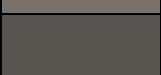
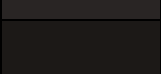
5.2 Color y uso tipográfico del color

Para la composición de los colores de nuestra web, hemos seleccionado varios colores de la escala por defecto de TailwindCSS para garantizar la armonía y el contraste. A continuación, mostraremos los colores utilizados en nuestro proyecto con una muestra de color, nombre de la clase de TailwindCSS, código hexadecimal y el uso que le hemos dado:

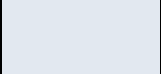
5.2.1 Blue – Acciones primarias y tema claro

Muestra	Clase	Hex	Uso
	blue-200	#BFDDBFE	Fondos hover, variante suave
	blue-400	#60A5FA	Acentos
	blue-500	#3B82F6	Acciones principales, focus
	blue-600	#2563EB	Botones primarios
	blue-800	#1E40AF	Contraste, textos sobre fondos claros

5.2.2 Stone – Colores neutros y tema claro

Muestra	Clase	Hex	Uso
	stone-100	#F5F5F4	Fondos suaves
	stone-200	#E7E5E4	Fondos base, bordes
	stone-300	#D6D3D1	Bordes, placeholders
	stone-400	#A8A29E	Iconos secundarios
	stone-500	#78716C	Textos secundarios
	stone-600	#57534E	Textos medios
	stone-800	#292524	Textos principales
	stone-900	#1C1917	Textos oscuros

5.2.3 Slate – Colores neutros y tema oscuro

Muestra	Clase	Hex	Uso
	slate-50	#F8FAFC	Textos sobre fondos oscuros
	slate-200	#E2E8F0	Bordes claros
	slate-400	#94A3B8	Iconos secundarios
	slate-500	#64748B	Textos secundarios
	slate-700	#334155	Bordes, fondos
	slate-800	#1E293B	Fondos de tarjetas
	slate-900	#0F172A	Fondo principal
	slate-950	#020617	Fondos muy oscuros

5.2.4 Cyan – Acciones primarias y tema oscuro

Muestra	Clase	Hex	Uso
	cyan-200	#A5F3FC	Hover suave
	cyan-400	#22D3EE	Acentos
	cyan-500	#06B6D4	Acciones principales
	cyan-600	#0891B2	Botones primarios
	cyan-800	#155E75	Contraste

5.2.5 Colores semánticos

Muestra	Clase	Hex	Uso
	red-500	#EF4444	Errores
	red-600	#DC2626	Peligro, eliminar, cerrar
	green-500	#22C55E	Éxito, confirmar

5.3 Tipografía

La fuente principal que hemos usado en el proyecto es la fuente Roboto de Google Fonts en diferentes variables:

Familia	Pesos	Subset	Uso
Roboto-sans	400	latin	Texto principal, títulos, body
	Estos es Roboto en peso 400		
	700	latin	
	Esto es Roboto en peso 700		
Robot Flex	500	latin	Acentos, énfasis secundarios
	Esto es Roboto Flex en peso 500		

Estas fuentes se cargan desde Google Fonts vía Next.js en el archivo 'app/fonts.ts'.

```
// app/fonts.ts
export const robotoSans = Roboto({ subsets: ["latin"], weight: ["400", "700"] });
export const robotoFlex = Roboto_Flex({ weight: ["500"], subsets: ["latin"] });
```

En cuanto al tamaño de letra, por todo el proyecto hemos usado los siguientes tamaños:

Etiqueta HTML	Clase	Tamaño	Ejemplo	Uso
H1	text-4xl	2.25rem (36px)	Ejemplo	Títulos principales
H2	text-3xl	1.875rem (30px)	Ejemplo	Secciones
H3	text-2xl	1.5rem (24px)	Ejemplo	Subsecciones
H4	text-xl	1.25rem (20px)	Ejemplo	Subtítulos
body	text-base	1rem (16px)	Ejemplo	Texto normal
small	text-sm	0.875rem (14px)	Ejemplo	Texto secundario, etiquetas

5.4. Iconografía

Para nuestra Iconografía hemos usado la librería Font Awesome Icons. Usamos esta librería porque dispone de una gran colección de iconos de uso gratuito que nos permite usar en nuestra aplicación comercial sin regalías ni cánones. Además, tiene una gran integración y optimización con nuestro stack tecnológico.

Los paquetes que usamos en nuestro proyecto son:

```
"@fortawesome/fontawesome-svg-core": "^7.1.0",
```

```
"@fortawesome/react-fontawesome": "^3.1.1",
"@fortawesome/free-solid-svg-icons": "^7.1.0",
"@fortawesome/free-brands-svg-icons": "^7.1.0",
"@fortawesome/free-regular-svg-icons": "^7.1.0"
```

Configuración

Para poder usar los iconos debemos añadir en nuestro root layout.tsx la siguiente dependencia:

```
import { config } from "@fortawesome/fontawesome-svg-core";
import "@fortawesome/fontawesome-svg-core/styles.css";

config.autoAddCss = false;
```

Listado de iconos usados

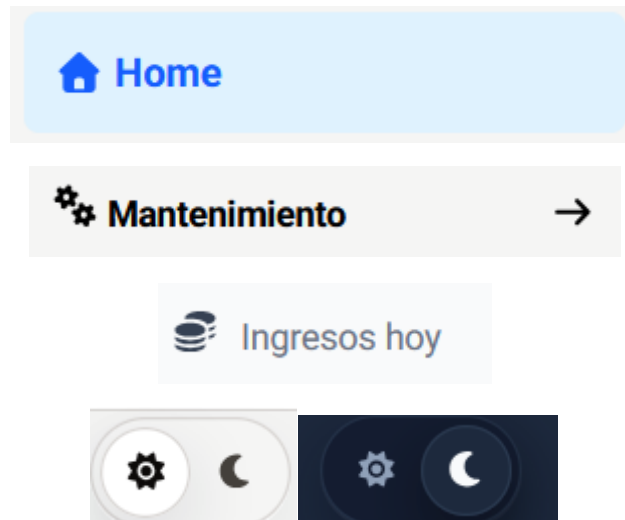
Icono	Muestra	Uso
faUser		Usuario, login
faLock		Contraseña, seguridad
faChartLine		Dashboard, métricas
faTicket		Tickets
faCoins		Monetización
faReceipt		Facturas
faEye		Ver / visualizar
faPencil		Editar
faTrash		Eliminar
faSun		Elegir tema – modo claro
faMoon		Elegir tema – modo oscuro
faHouse		Link Inicio en dashboard
faInvoiceDollar		Links a cierres de caja
faGears		Links a sección mantenimiento

Tamaños

Los tamaños que hemos usados son “sm”, “lg”, “2x” y “5x” que se asigna en el atributo size de FontAwesomeIcon

```
<FontAwesomeIcon icon={faUser} size="lg" />
```

Muestras



5.5 Estructura de la web. Maquetación - Wireframes

En esta sección vamos a mostrar los wireframes de las secciones más importantes.

5.5.1 Dashboard

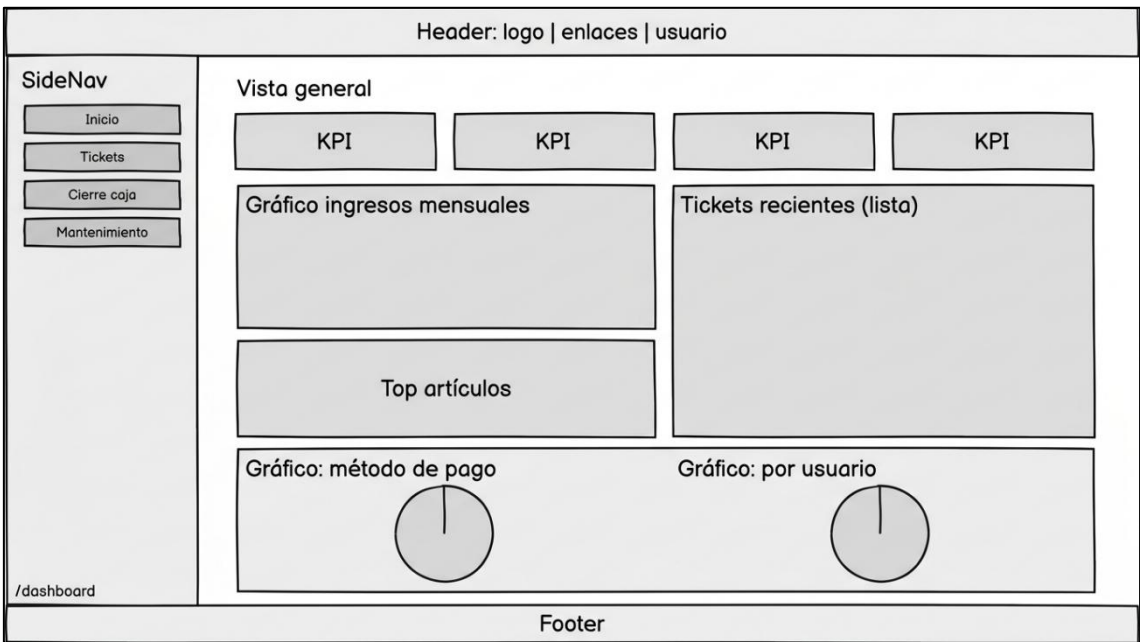


Ilustración 5. Wireframe lowfidelity "/dashboard"

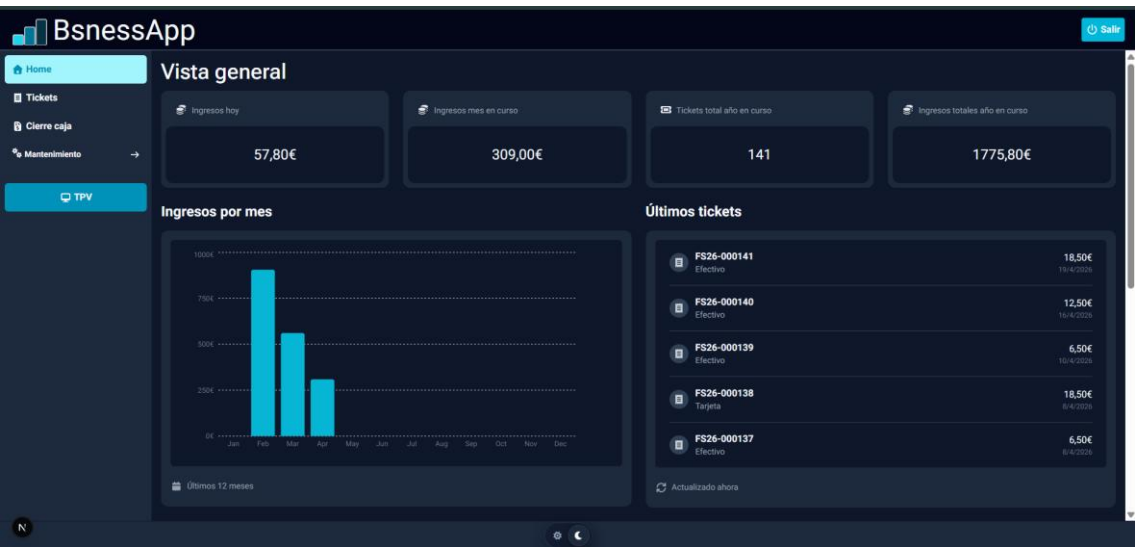


Ilustración 6. Pantalla final "/dashboard"

5.5.2 Pos

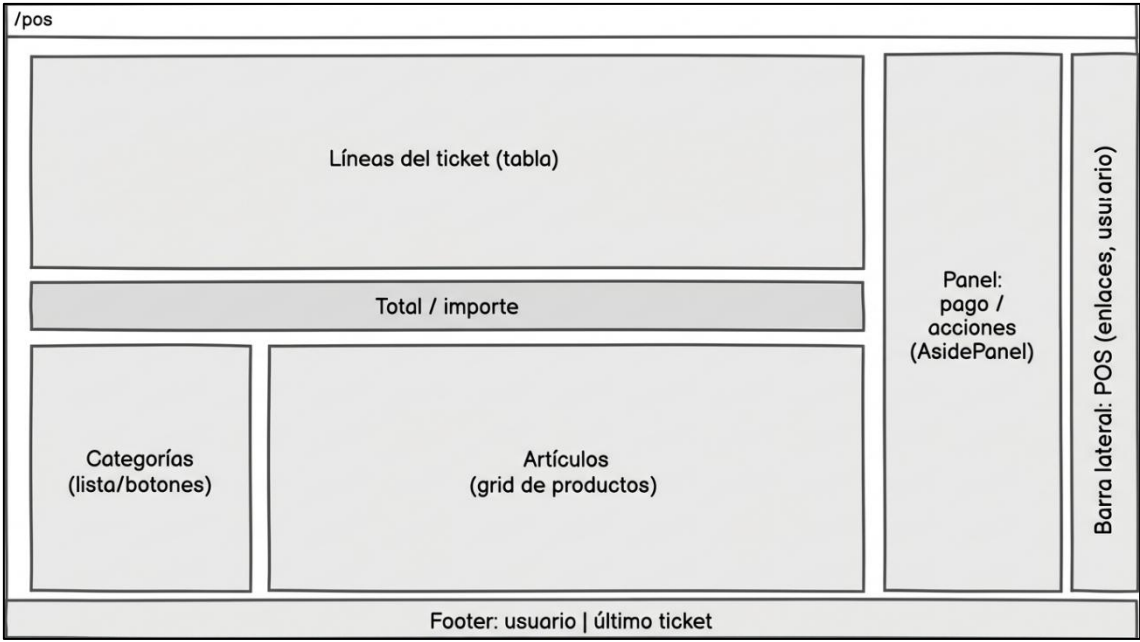


Ilustración 7. Wireframe lowfidelity "/pos"

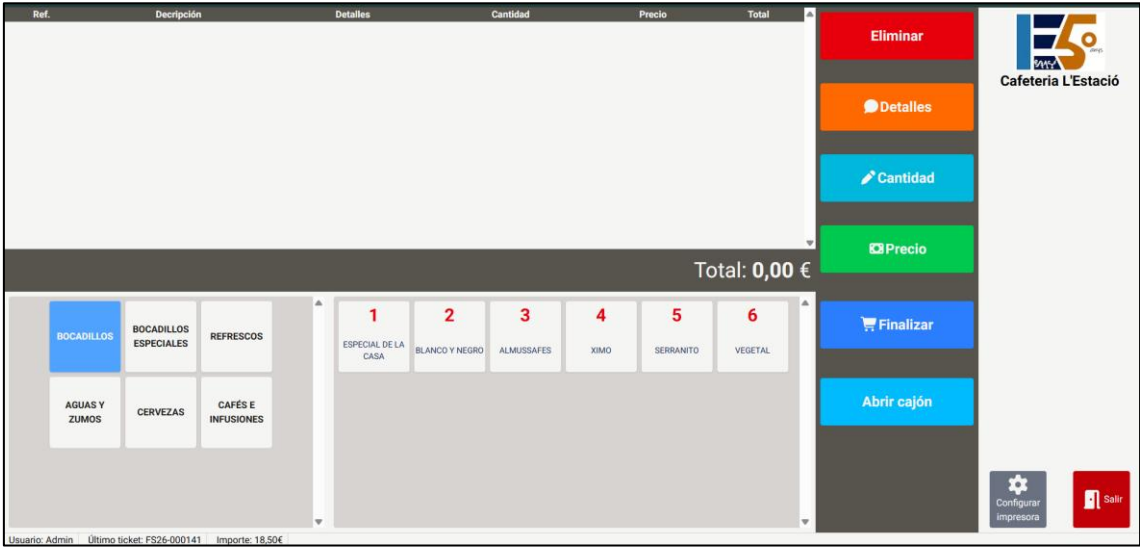


Ilustración 8. Pantalla final "/pos"

5.6 Espaciados y layout

En esta sección definiremos cómo estructuramos el contenido en la página: ancho y centrado con contenedores (container, max-w-*) y criterios de uso según el tipo de bloque. Mostraremos la metodología de **espaciado** usado con utilidades de *Tailwind*:

- gap entre elementos
- p (padding) y px para relleno y variaciones en breakpoints

Incluye la **tabla de breakpoints** (sm, md, lg, xl, 2xl) y **patrones de grid** frecuentes (columnas, sidebar, etc.) para diseños responsivos coherentes.

5.6.1 Contenedores

Los contenedores son cajas de diseño que limitan el ancho del contenido y añaden márgenes internos, para que el texto y los bloques no se peguen a los bordes de la pantalla. *container mx-auto px-4* centra el bloque; *max-w-** fija un ancho máximo distinto según el caso (páginas anchas, formularios, modales, etc.). Así el *layout* se mantiene legible y coherente en monitores grandes y pequeños. Dentro de los estilos de Tailwind, aquí un listado de los que hemos usado:

Clase	Uso
container mx-auto px-4	Contenedor centrado con padding lateral
max-w-7xl	Ancho máximo ~80rem (1280px)
max-w-3xl	Ancho máximo 48rem (768px) - Formularios, textos largos
max-w-2xl	Ancho máximo 42rem (672px) - Modales, secciones estrechas
max-w-lg	Ancho máximo 32rem (512px) - Formularios compactos
max-w-sm	Ancho máximo 24rem (384px) - Inputs, tarjetas pequeñas

5.6.2 Espaciado

El espaciado hace a las clases de Tailwind que usamos para **separar** y **rellenar** contenido: gap entre flex/grid, p-* y px-* para márgenes internos. Sirve de referencia rápida para no inventar escala en cada componente y mantener espaciado coherente en toda la app. A continuación, un listado con las clases típicas usadas en el proyecto:

Tipo	Clases típicas
Gaps	gap-1, gap-2, gap-4, gap-6, gap-8
Padding	p-2, p-3, p-4, p-6, p-8
Padding lateral	px-4, px-5, md:px-3

6. Caso de uso

Esto es un caso inventado, cualquier similitud con la realidad es pura casualidad.

El instituto IES l'Estació, con la renovación de su edificio, está buscando un software TPV para la gestión de su cafetería. En esta demo, mostramos el flujo de trabajo para el uso de un TPV para una cafetería desde la creación de tickets, cierres de caja diarios y diferentes interfaces según rol del usuario (administrador/usuario_normal).

Las funcionalidades que tenemos activas para la demo son:

- Sistema de autenticación de usuarios y establecimiento de roles de usuario.
- Dashboard con datos estadísticos del funcionamiento de la actividad económica.
- Creación de artículos, modificación y eliminación.
- Creación de categorías, modificación y eliminación.
- Creación de usuarios, modificación y eliminación.
- Creación de tipos impositivos, modificación y eliminación.
- Generación de informes en pdf.
- Generación de tickets en pdf y posibilidad de enviar vía email.
- Uso totalmente funcional de la pantalla principal del TPV.
- Uso de impresora física para imprimir en papel el ticket.
- Posibilidad de abertura de cajón portamonedas.

7. Anexos

7.1 Anexo 1 – Diagrama de clase

Ver archivo Anexo 1 - Diagrama de clase.

7.2 Anexo 2 – Diagrama entidad relación

Ver archivo Anexo 2 - Diagrama entidad relación.

7.3 Anexo 3 - CU-001_Registrar_Venta_POS

Ver archivo Anexo 3 - CU-001_Registrar_Venta_POS.

8. Bibliografía

- **Documentación TPV/POS:**
 - o Odoo POS documentation:
https://www.odoo.com/documentation/19.0/applications/sales/point_of_sale.html
 - o React thermal printer dependency:
<https://github.com/seokju-na/react-thermal-printer>
 - o Escpos Project:
<https://github.com/lsongdev/node-escpos>
 - o Node serial port:
<https://github.com/serialport/node-serialport>
 - o Node Thermal Printer
<https://github.com/Klemen1337/node-thermal-printer>
- **Documentación Next.js:**
 - o Next.js docs:
<https://nextjs.org/docs>
 - o Next.js Learn:
<https://nextjs.org/learn>
- **Documentación React:**
 - o <https://react.dev/learn>
 - o <https://react.dev/reference/react>
- **Documentación Tailwind CSS:**
 - o <https://tailwindcss.com/>
- **Documentación Font Awesome Icons:**
 - o Como usar los iconos en Next.js:
<https://docs.fontawesome.com/web/use-with/react/use-with>
- **Documentación Drizzle ORM:**
 - o <https://orm.drizzle.team/docs/overview>
 - o Migraciones:
<https://orm.drizzle.team/docs/kit-overview>
 - o Conexión con NeonDB
<https://orm.drizzle.team/docs/get-started/neon-new>
- **Documentación generación de email con Nodemailer:**
 - o Documentación general:
<https://nodemailer.com/>
 - o Adjuntar archivos:
<https://nodemailer.com/message/embedded-images>
 - o
- **Configuración DNS en Vercel para uso de email Ionos:**
 - o <https://vercel.com/docs/domains/managing-dns-records>
 - o <https://www.ionos.es/ayuda/dominios/configurar-un-servidor-de-correo-y-registros-relativos/utilizar-un-dominio-con-servidores-de-correo-de-otros-proveedores-ajustar-registros-mx/>
 - o <https://www.ionos.es/ayuda/correo/solucion-de-problemas-correo-basico/correo-profesional/modificar-los-registros-mx-para-la-recepcion-de-correo-electronico-con-los-servidores-de-correo-ionos/>